

Relatório - Trabalho 1 - Inteligência Artificial

Membros do grupo:

- Adriel de Souza (00579100)
- Arthur Chagas Bridi (00585225)
- Rafael Stephanou (00590367)

Comparação experimental (Tarefas 1-4):

```
# Comandos executados para obter os resultados:
python3 pacman.py -l mediumMaze -p SearchAgent -a fn=depthFirstSearch
python3 pacman.py -l mediumMaze -p SearchAgent -a fn=breadthFirstSearch
python3 pacman.py -l mediumMaze -p SearchAgent -a fn=uniformCostSearch
python3 pacman.py -l mediumMaze -p SearchAgent -a fn=aStarSearch,heuristic=manhattanHeuristic

python3 pacman.py -l bigMaze -p SearchAgent -a fn=depthFirstSearch
python3 pacman.py -l bigMaze -p SearchAgent -a fn=breadthFirstSearch
python3 pacman.py -l bigMaze -p SearchAgent -a fn=uniformCostSearch
python3 pacman.py -l bigMaze -p SearchAgent -a fn=aStarSearch,heuristic=manhattanHeuristic
```

Resultados - mediumMaze:

Algoritmo	Custo da Solução	Nós Expandidos
Depth First Search	130	146
Breadth First Search	68	269
Uniform Cost Search	68	269
A* Search (Manhattan Heuristic)	68	221

Resultados - bigMaze:

Algoritmo	Custo da Solução	Nós Expandidos
Depth First Search	210	390
Breadth First Search	210	620
Uniform Cost Search	210	620
A* Search (Manhattan Heuristic)	210	549

Análise dos Resultados:

Os testes realizados demonstram as características fundamentais de cada algoritmo de busca.

Em relação à otimalidade, os algoritmos BFS, UCS e A* garantiram o caminho ótimo em ambos os labirintos. O DFS, por outro lado, apresentou uma solução sub-ótima no `mediumMaze` (custo 130 contra 68 do ótimo), o que evidencia que sua estratégia de busca não leva em conta o custo acumulado do caminho. No `bigMaze`, embora o DFS tenha coincidido com o custo ótimo de 210, isso decorre da estrutura específica do mapa ou da ordem de expansão, não sendo uma garantia teórica do algoritmo.

Observou-se que o BFS e o UCS apresentaram resultados idênticos em termos de nós expandidos e custo final. Isso ocorre porque o problema do Pac-Man possui custos de transição uniformes (valor 1 para cada movimento), fazendo com que a busca de custo mínimo do UCS se comporte exatamente como a busca em largura do BFS.

O impacto da heurística no algoritmo A* foi significativo para a eficiência da busca. Ao utilizar a distância de Manhattan, o A* conseguiu encontrar o caminho ótimo expandindo uma quantidade sensivelmente menor de nós que o BFS e o UCS (por exemplo, 549 contra 620 no `bigMaze`). Isso demonstra como a incorporação de conhecimento sobre o objetivo permite direcionar a exploração e descartar caminhos menos promissores.

Por fim, quanto à eficiência de expansão, o DFS expandiu a menor quantidade de nós em ambos os cenários, devido à sua natureza de explorar profundamente um ramo antes de retroceder. Contudo, essa rapidez de processamento sacrifica a qualidade da solução final, tornando o DFS inadequado quando o objetivo principal é

minimizar o trajeto percorrido pelo Pac-Man.

Modelagem de estado (Corners/Food) (Tarefa 5)

Corners:

A modelagem dos estados para o problema dos cantos foi definida como uma tupla contendo a posição atual do Pac-Man e o status de visitação dos quatro cantos: $((x, y), (c1, c2, c3, c4))$.

- **Posição:** Uma tupla (x, y) com as coordenadas inteiras do Pac-Man.
- **Status dos cantos:** Uma tupla de quatro valores booleanos, onde cada posição corresponde a um dos cantos do mapa. O valor `True` indica que o canto já foi visitado, e `False` caso contrário.

O estado inicial é composto pela posição de partida e a tupla $(False, False, False, False)$. O estado objetivo é alcançado quando todos os elementos da tupla de status são `True`, indicando que os quatro cantos foram visitados. Na função de sucessores, a cada movimento, verifica-se se a nova posição coincide com algum dos cantos; se sim, o booleano correspondente na tupla é atualizado para `True`.

Food:

A modelagem dos estados para o Food Problem é uma tupla contendo a posição atual do Pac-Man e uma matriz 2D de comida restante: $((x, y), foodGrid)$.

- **Posição:** Uma tupla (x, y) com as coordenadas inteiras do Pac-Man.
- **Grid de comida:** Um objeto Grid (matriz 2D) de valores booleanos onde `True` indica que ainda há comida naquela posição e `False` indica que já foi consumida.

O estado inicial é composto pela posição de partida e o grid completo de comida (todos os pontos `True`). O estado objetivo é alcançado quando não há nenhum `True` restante no grid, ou seja, todas as comidas foram coletadas. Na função de sucessores, a cada movimento, verifica-se se a nova posição contém comida; se sim, o grid é copiado para o próximo estado e a posição correspondente é marcada como `False`.

Heurísticas (admissibilidade + impacto) (Tarefas 6-7)

Tarefa 6 - Corners Heuristic:

A heurística implementada para o problema dos cantos calcula a distância de Manhattan entre a posição atual do Pac-Man e cada um dos cantos ainda não visitados, retornando a maior dessas distâncias: $h(s) = \max\{Manhattan(pos, c_i) \mid c_i \in C_{\{unvisited\}}\}$.

Essa heurística é admissível porque, para atingir o estado objetivo, o Pac-Man precisa obrigatoriamente visitar todos os cantos restantes. Como ele só pode se mover uma casa por vez (movimentos ortogonais), o custo real para chegar ao canto mais distante será sempre maior ou igual à distância de Manhattan até ele, mesmo ignorando as paredes do labirinto. Portanto, a heurística nunca superestima o custo real para alcançar o objetivo, garantindo a admissibilidade.

```
# Comandos para análise do impacto da heurística:

# Sem heurística:
python3 pacman.py -l mediumCorners -p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=nullHeuristic -z 0.5
python3 pacman.py -l bigCorners -p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=nullHeuristic -z 0.5

# Com heurística:
python3 pacman.py -l mediumCorners -p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic -z 0.5
python3 pacman.py -l bigCorners -p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic -z 0.5
```

Algoritmo	Labirinto	Custo da Solução	Nós Expandidos
A* Search (Sem Heurística)	mediumCorners	106	1699
A* Search (Corners Heuristic)	mediumCorners	106	1136
A* Search (Sem Heurística)	bigCorners	162	7949
A* Search (Corners Heuristic)	bigCorners	162	4380

Os resultados evidenciam um impacto significativo da heurística na eficiência da busca. No labirinto `mediumCorners`, a utilização da heurística reduziu o número de nós expandidos de 1699 para 1136, representando uma diminuição de aproximadamente 33%. Já no `bigCorners`, a redução foi ainda mais expressiva, com os nós expandidos caindo de 7949 para apenas 4380, o que corresponde a uma redução de cerca de 45%.

Tarefa 7 - Food Heuristic:

A heurística implementada para o Food Problem calcula a distância real no labirinto (`mazeDistance`) entre a posição atual do Pac-Man e cada uma das comidas restantes, retornando a maior dessas distâncias: $h(s) = \max\{mazeDistance(pos, f_i) \mid f_i \in F_{\{remaining\}}\}$

A heurística é admissível porque o Pac-Man precisa obrigatoriamente visitar toda a comida, incluindo a mais distante. O custo real da solução nunca será menor que a distância até ela, independente do caminho tomado. Usar mazeDistance em vez de Manhattan torna a heurística mais precisa porque considera as paredes, reduzindo o número de nós expandidos pelo A*.

```
# Comandos para análise do impacto da heurística:

# Sem heurística:
python3 pacman.py -l trickySearch -p SearchAgent -a fn=aStarSearch,prob=FoodSearchProblem,heuristic=nullHeuristic -z 0.5

# Com heurística:
python3 pacman.py -l trickySearch -p SearchAgent -a fn=aStarSearch,prob=FoodSearchProblem,heuristic=foodHeuristic -z 0.5
```

Algoritmo	Custo da Solução	Nós Expandidos
A* Search (Sem Heurística)	60	16688
A* Search (foodHeuristic)	60	4137

A utilização da heurística reduziu o número de nós expandidos de 16688 para 4137, representando uma redução de aproximadamente 75%, mantendo o custo ótimo da solução em 60 passos nos dois casos.

Declaração do uso de IA

Durante o desenvolvimento do projeto, foram utilizadas IAs generativas como ferramenta de apoio, principalmente como meio de tirar dúvidas conceituais, auxiliar no debug de códigos e polir a escrita do relatório. Todas as funções criadas foram desenvolvidas pelos membros do grupo.